

Today's Content

- ① Subarray Basics
- ② Printing subarray
- ③ Generating all subarrays
- ④ Printing all subarray sums
 - Approach 1, Approach 2.
 - Max subarray sum
 - Sum of all subarray sums
 - Approach 2
- ⑤ Max subarray sum with $\text{len} = K$.

// Subarray Basics

- 1) Continuous part of an array is called a subarray.
- 2) Single element is also a subarray
- 3) Complete array is also a subarray.

e.g. 0 1 2 3 4 5 6 7 8 9
 -2 4 6 3 8 1 4 3 2 -10

indices:- {3, 4, 5, 7, 8} → Not a subarray. Not continuous.

indices:- {4, 5, 6, 7, 8} → [4-8] → subarray

Indices:- [2 6] → {2, 3, 4, 5, 6} → subarray.
 ↓ ↓
 si ei

```
// printSubArray (int arr[], s, e)
// Subarray [s, e]
for (int i = s; i <= e; i++) {
    |   print(arr[i]);
    |
}
}
```

↑ subarray start index
→ subarray end index

T.C = $O(N)$
S.C = $O(1)$

// How many subarrays

e.g. 1:- arr[4] = ⁰-1 ¹3 ²7 ³9

0-0 :- [-1]

0-1 :- [-1, 3]

0-2 :- [-1, 3, 7]

0-3 :- [-1, 3, 7, 9]

↓
4

1-1 :- [3]

1-2 :- [3, 7]

1-3 :- [3, 7, 9]

↓
3

2-2 :- [7]

2-3 :- [7, 9]

↓
2

3-3 :- [9]

↓
1

$$\text{total subarrays} = 4 + 3 + 2 + 1 = 10$$

↓
sum of first 4
natural numbers.

Generalise:-

Given N elements, how many subarrays.

arr[N] = { 0, 1, 2, 3, 4, 5, ..., n, n+1, ..., N-2, N-1 }

Starting at 0:- starting at 1:- starting at 2:- ... starting at (N-2):- starting at N-1

0-0

0-1

0-2

0-3

0-4

⋮

0-N-1

N subarrays

1-1

1-2

1-3

1-4

⋮

1-N-1

N-1 subarrays

2-2

2-3

2-4

2-5

⋮

2-N-1

N-2 subarrays

N-2, N-2

N-2, N-1

2 subarrays

N-1, N-1

1 subarray

$$\text{Total sub arrays} = (N) + (N-1) + (N-2) + (N-3) + \dots + 2 + 1$$

$$= \frac{N(N+1)}{2}$$

→ sum of N natural numbers

// Print All Subarrays-

e.g. $N=3$
arr[3] = { 8, 2, 9 }

0-0 : [8] $\rightarrow$ 1n
0-1 : [8, 2] $\rightarrow$ 1n
0-2 :- [8, 2, 9] $\rightarrow$ 1n
1-1 :- [2] $\rightarrow$ 1n
1-2 :- [2, 9] $\rightarrow$ 1n
2-2 :- [9]

index positions where your subarray can start
 $\uparrow$
for (s=0; s < N; s++) {
 for (e=s; e < N; e++) {
 // [s, e] is a subarray.
 for (k=s; k <= e; k++) {
 print(arr[k]);
 }
 print("\n");
 }
}

fixing your end index

T.C. = $O(N^3)$

// Print all subarray sums.

```
for (s=0; s<N; s++) {
```

```
    for (e=s; e<N; e++) {
```

```
        // [s,e] is a subarray.
```

```
        int sum=0;
```

```
        for (k=s; k<=e; k++) {
```

```
            sum += arr[k];
```

```
        }  
        print(sum);
```

```
    }  
}
```

T.C = $O(N^3)$

S.C = $O(1)$

→ You can optimise using `pfsum[]`

* `pf[]` → `[s,e]`

todo

→ `pf[e] - pf[s-1]` (optimise using `pfsum[]`, range sum queries)

↓
Becomes $O(1)$

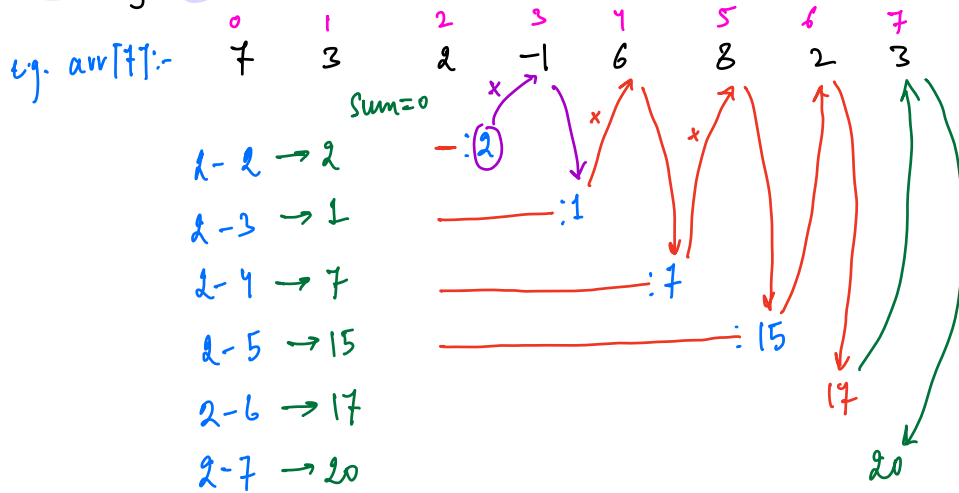
New T.C ⇒ $O(N + N^2)$

⇒ $O(N^2)$

S.C ⇒ $O(N)$

↓
can you optimise more?

// Printing all subarray sums starting at index 2



Carry forward.

L → R

Sum = 0;

```

for (j = 2; j < N; j++) {
    sum = sum + arr[j];
    print(sum);
}
  
```

Dry run :- N = 8

j	sum = 0
2	2
3	1
4	7
...	...

// Printing all subarray sums using carry forward.

for (i = 0; i < N; i++) {

Sum = 0;

```

for (j = i; j < N; j++) {
    sum = sum + arr[j];
    print(sum);
}
  
```

→ T.C = $O(N^2)$

→ S.C = $O(1)$

// Get max of all subarray sums.

maxSum = arr[0];

for (i = 0; i < N; i++) {

sum = 0;

for (j = i; j < N; j++) {

sum = sum + arr[j]

maxSum = max(maxSum, sum);

}

print(maxSum);

→ maximum sum out of all subarray sums.

↓
taking the max of curr sum and prev max and storing in max.

T.C = $O(N^2)$ → optimise this to $O(N)$

S.C = $O(1)$

↓
KADANE'S ALGO → Advance Batch.

Break → 10:42 PM.

// Print sum of all subarray sums.

e.g. 1:- arr[4] = $\begin{matrix} 0 & 1 & 2 & 3 \\ \{-1 & 3 & 7 & 9\} \end{matrix}$

0-0 :- [-1] :- -1 1-1 :- [3] :- 3 2-2 :- [7] :- 7 3-3 :- [9] :- 9
0-1 :- [-1, 3] :- -2 1-2 :- [3, 7] :- 10 2-3 :- [7, 9] :- 16
0-2 :- [-1, 3, 7] :- 9 1-3 :- [3, 7, 9] :- 19
0-3 :- [-1, 3, 7, 9] :- 18

Sum of all subarray sums = $-1 + 2 + 9 + 18 + 3 + 10 + 19 + 7 + 16 + 9$
= 92 $\rightarrow$ final Ans you need to print.

totalSum = 0;

for(i=0; i<N; i++) {

sum = 0;

for(j=i; j<N; j++) {

sum = sum + arr[j]

totalSum += sum;

}

}

print(totalSum);

T.C = $O(N^2)$

S.C = $O(1)$

Using same piece of code

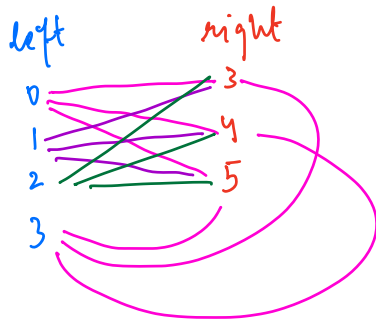
- ① print all subarray sums.
- ② get max subarray sum
- ③ get totalSum of all subarrays.

Q: In how many subarrays index 3 is present?

arr:-

0	1	2	3	4	5
3	-2	4	-1	2	6

start indices end indices



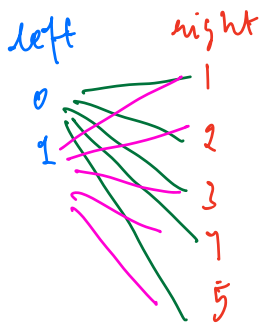
total subarrays with index 3 present
 $\Rightarrow 4 * 3 \Rightarrow 12$

Quiz In how many subarrays index 1 is present?

arr:-

0	1	2	3	4	5
3	-2	4	-1	2	6

start indices end indices



Ans $\Rightarrow 2 * 5$
 $\Rightarrow 10$

Quiz In how many subarrays index 0 is present?

arr:- 0 1 2 3 4 5
 3 -2 4 -1 2 6

left	right
0	0
	1
	2
	3
	4
	5

$$\text{Ans} = 1 * 6 = 6$$

Generalise:-

index i is present in how many subarrays?

left:- $[0, i] \rightarrow i+1$

right:- $[i, N-1] \rightarrow (N-1) - i + 1$
 $\rightarrow N - i - i + 1$
 $\rightarrow N - i$

total no. of subarrays containing i^{th} index

$$\Rightarrow (i+1)(N-i)$$

$N=6$

	0	1	2	3	4	5
arr[]:-	3	-2	4	-1	2	6
left =	1	2	3	4	5	6
right =	6	5	4	3	2	1
total no. of subarrays containing this element =	6	10	12	12	10	6
total sum of all subarray sums.	18	-20	48	-12	20	36

e.g.

	0	1	2	3
arr[]:-	-1	3	7	9
left:-	1	2	3	4
right:-	4	3	2	1
total subarray:-	4	6	6	4
total sum :-	-4	+ 18	+ 42	+ 36

$\Rightarrow 92$ ✓ Hence, verified.

// Pseudo Code:-

sum = 0;

for (i = 0; i < N; i++) {

left = i + 1;

right = N - i;

count = left * right;

sum = sum + (count * arr[i]);

}
return sum;

T.C = $O(N)$

S.C = $O(1)$

→ Contribution technique

Double:-

arr[] = [1, 2, 3, 6, 9]
 3 6 12 21

arr[i] = arr[i-1] + arr[i].

arr[] = 1, 3, 6, 12, 21